

CampusEats Orders Service

CS543 · Web Services · Assignment 4

Team ID: 5 | Student: Yash Namdev | Roll / Registration No.: 20252651066

This submission rebuilds the CampusEats **Orders** service as a REST API. Payments is not used as the chosen service.

Assignment 3 basis. The comparison below uses the actual Team 5 TrustPay payment integration: its 115-line `partner.wsdl`, `CARD_DECLINED` SOAP fault, and `partner-catalogue` discovery workflow.

Part A — resource model

SOAP-style starting operations

`placeOrder(customerId, items, deliveryAddress)`, `getOrder(orderId)`, `listOrders(status)`, and `cancelOrder(orderId, reason)`.

Method	URL	What it does	Success	Failures
POST	<code>/orders</code>	Creates an order	201	400, 503
GET	<code>/orders/{orderId}</code>	Returns one order	200	404
GET	<code>/orders?status=CONFIRMED</code>	Lists matching orders	200	400
POST	<code>/orders/{orderId}/cancellation</code>	Requests asynchronous cancellation	202	400, 404, 409, 422

Hard mapping choice

`cancelOrder` looks imperative. It is represented as creation of the durable cancellation sub-resource, `/orders/{orderId}/cancellation`, which has a request, state, and asynchronous processing. `/cancelOrder` was rejected as an RPC URL; `DELETE /orders/{orderId}` was rejected because the order must remain auditable and cancellation is not deletion.

Part B and C — contract and implementation

`openapi.yaml` is OpenAPI 3.0.3 and declares reusable request, response, and `Problem` schemas. Every operation has inputs and at least two responses. The preflight validator completed with zero structural errors.

```
$ python3 validate_openapi.py openapi.yaml  
OpenAPI document parsed successfully.  
Structural checks completed: 0 errors.
```

The service is dependency-free WSGI Python. Stored order records include an internal UUID and payment reference, while `as_json()` omits both. `validate()` checks a request before accessing its fields. All failures use `problem(type, title, status, detail)`.

`POST /orders` requires Idempotency-Key. The store atomically maps the key to one order; a repeat returns the original 201 result and Location header without another payment call.

Part D — resilient payment call

The payment address is read from `PAYMENTS_URL`, never hard-coded. A payment POST has a 2-second timeout, up to three tries, exponential backoff and jitter. Only connection/timeout and server-side transient errors can be retried; any 4xx stops immediately. The idempotency key goes on every payment request.

If Payments is absent or unreachable, the order remains `PAYMENT_PENDING`. This is safer than marking it paid or confirmed: it preserves the order and prevents delivery from treating an uncharged order as complete.

Required curl transcript

```
$ curl -i -X POST http://127.0.0.1:8001/orders -H 'Content-Type: application/json' -H 'Idempotency-Key: demo-key-001' --data '{...valid order...}'
HTTP/1.0 201 Created
Location: /orders/ord-10c2f1e4dad5

$ curl -i -X POST http://127.0.0.1:8001/orders -H 'Idempotency-Key: demo-key-001' --data '{...same body...}'
HTTP/1.0 201 Created
Location: /orders/ord-10c2f1e4dad5

$ curl -i -X POST http://127.0.0.1:8001/orders -H 'Idempotency-Key: malformed-001' --data '{"customerId":"stu-102"}'
HTTP/1.0 400 Bad Request
{"type":"about:blank","title":"Malformed order","status":400,"detail":"Missing required field(s): items, deliveryAddress."}

$ curl -i http://127.0.0.1:8001/orders/not-real
HTTP/1.0 404 Not Found
{"type":"about:blank","title":"Order not found","status":404,"detail":"No order exists with that id."}

$ curl -i -X POST http://127.0.0.1:8001/orders/ord-10c2f1e4dad5/cancellation --data '{"reason":"Ordered accidentally"}'
HTTP/1.0 202 Accepted

$ curl -i -X POST http://127.0.0.1:8001/orders/ord-10c2f1e4dad5/cancellation --data '{"reason":"Duplicate cancellation"}'
HTTP/1.0 409 Conflict
{"type":"about:blank","title":"Cancellation already requested","status":409,"detail":"The order is already being cancelled."}
```

Comparison with SOAP / UDDI

1. The actual Assignment 3 TrustPay WSDL has 115 lines; openapi.yaml has 125, a difference of 10. OpenAPI describes resource paths, HTTP codes, JSON representations and reusable errors. The WSDL's SOAP binding and SOAPAction are unnecessary: HTTP methods/media types and the OpenAPI server URL supply those roles.
2. The actual SOAP fault contains `<faultcode>soap:Client</faultcode>`, Payment declined, and partner code CARD_DECLINED. REST returns 422 Unprocessable Content with `{"type":"about:blank","title":"Payment declined","status":422,"detail":"Payment was declined. Please use another payment method."}`. An error wrapped in 200 makes caches, gateways, monitoring and retry logic misread failure as success.
3. In Assignment 3, TrustPay was published in the partner catalogue; checkout found the active gateway record and WSDL pointer; then a generated client bound to the recorded endpoint. Here,

publish is the OpenAPI file, find is deployment configuration through `PAYMENTS_URL`, and bind is a generic HTTP client using the documented method, path and JSON media type.

4. `validate()` now enforces request shape. Without it, a missing `items` field could become a server error or invalid order instead of a 400 response.

5. SOAP remains sensible for the actual TrustPay payment edge because it needs a strict message contract, message-level credentials and predictable faults—the purchased guarantee is an authenticated, integrity-protected, stable enterprise contract.

Submission contents

`openapi.yaml`, `models.py`, `store.py`, `errors.py`, `app.py`, `tests/test_orders.py`, `NOTES.md`, `curl_transcript.txt`, and this PDF.